API Security Risk Analysis Report

Assessment of JSONPlaceholder Test API

Prepared by: Kenny Ntsako Kosa
Cyber Security Intern - Future Interns
Date: June 26, 2026

Executive Summary

This report presents a security risk analysis of the JSONPlaceholder API (jsonplaceholder.typicode.com), a widely used mock API for testing and prototyping. The assessment focused on authentication, authorization, data exposure, and information disclosure—critical components of modern SaaS security.

Key findings include: unauthenticated access to all user data, broken object-level authorization (IDOR) allowing access to any user's profile, excessive data exposure where the API returns massive datasets without pagination, and server information disclosure revealing the underlying technology stack.

Immediate remediation steps include implementing API key/token requirements, enforcing user-specific access controls, implementing pagination, and obfuscating server headers to reduce the attack surface.

Scope & Methodology

API Tested:

- https://jsonplaceholder.typicode.com – Mock data service

Scope of Assessment:

- Read-only GET requests
- No active exploitation or DoS testing
- Header and response inspection only

Tools Used:

- Browser (Chrome) – Request/Response Inspection
- Canva – Report Design

Ethics Statement: All tests were conducted strictly within ethical guidelines on publicly available test APIs. No production systems or sensitive data were accessed.

Risk Classification of Identified Vulnerabilities

#	Endpoint	Risk Identified	Severity	Business Impact	Remediation
1	GET /users	Unauthenticated Access – No API key/token required.	🔴 High	Attackers can retrieve the entire user database (names, emails, addresses) without any authentication, leading to mass data theft.	Require a valid API key or authentication token for all endpoints.
2	GET /users/{id}	Broken Object Level Authorization (IDOR) – Accessing other users' data.	🔴 High	By changing the ID in the URL (e.g., /users/1, /users/2), an attacker can view ANY user's personal details.	Implement server-side access controls. Users must only access their own data.
3	GET /users	Excessive Data Exposure – Returns all records at once.	🟡 Medium	The API returns 100+ records in a single request, increasing data leakage risk and server load.	Implement pagination (e.g., ?_limit=10&_page=1) to limit data returned per request.
4	Response Headers	Server Information Disclosure – Reveals tech stack.	🟢 Low	Headers reveal Express and heroku. Attackers can look for known exploits targeting these specific technologies.	Obfuscate or remove X-Powered-By header and customize the Server header.

Remediation Strategy

Remediation Roadmap (Priority Order)

Priority 1 – Immediate Action (High Risk)

- Implement Authentication: Enforce API keys or OAuth2 tokens for all /users endpoints to prevent public scraping.
- Enforce Authorization: Add server-side logic so that a user with ID 1 cannot request the data of user 2 (IDOR prevention).

Priority 2 – Short-Term Action (Medium Risk)

- Add Pagination: Modify /posts to accept _limit and _page parameters to prevent massive data dumps.

Priority 3 – Low-Risk Action

- Obfuscate Server Headers: Remove or customize the X-Powered-By and Server headers to hide the technology stack from attackers.

Figure 1: Unauthenticated access to the full user list. No API key or token required.

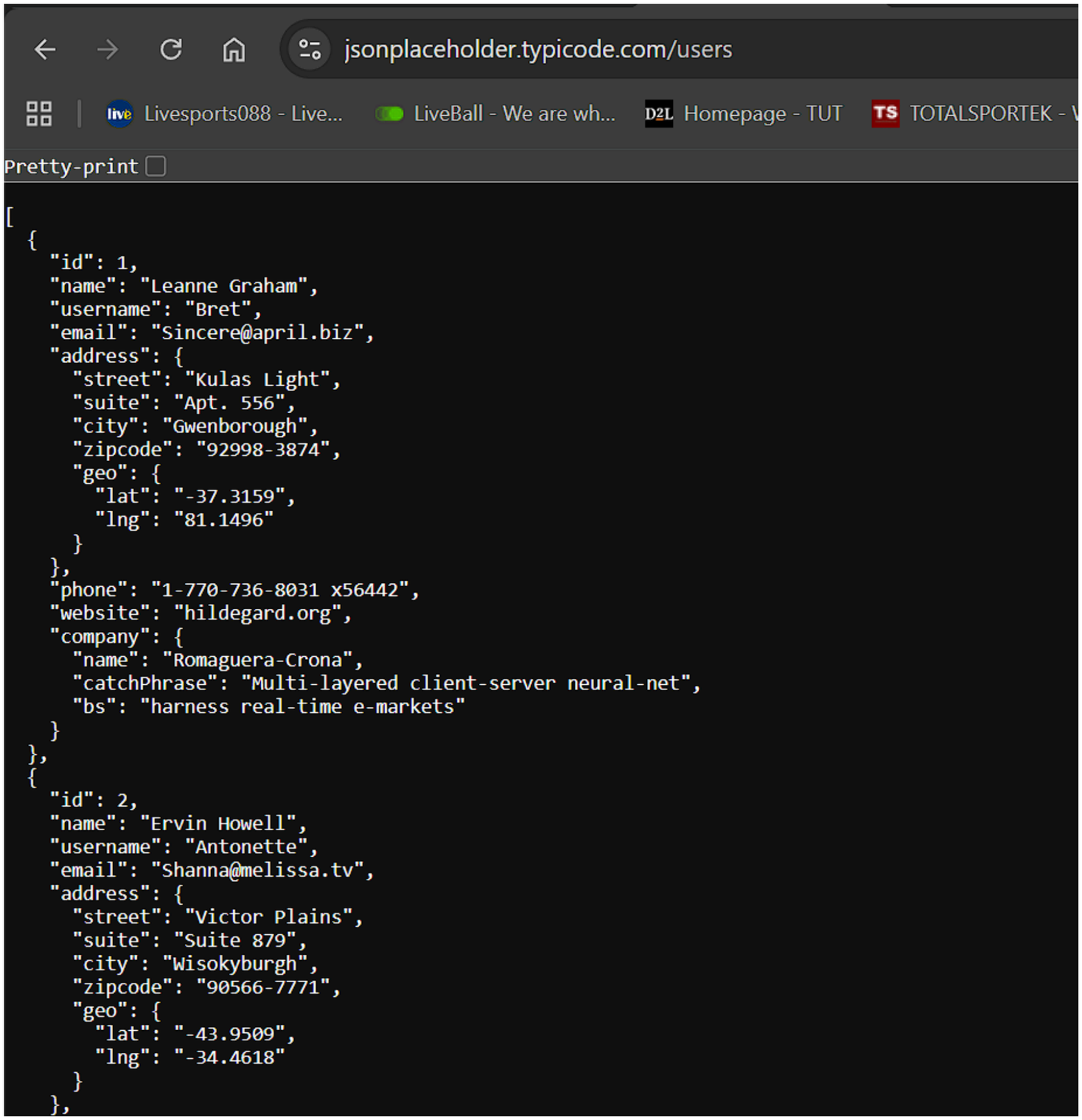


Figure 2: Accessing User 1's sensitive profile data (IDOR vulnerability).

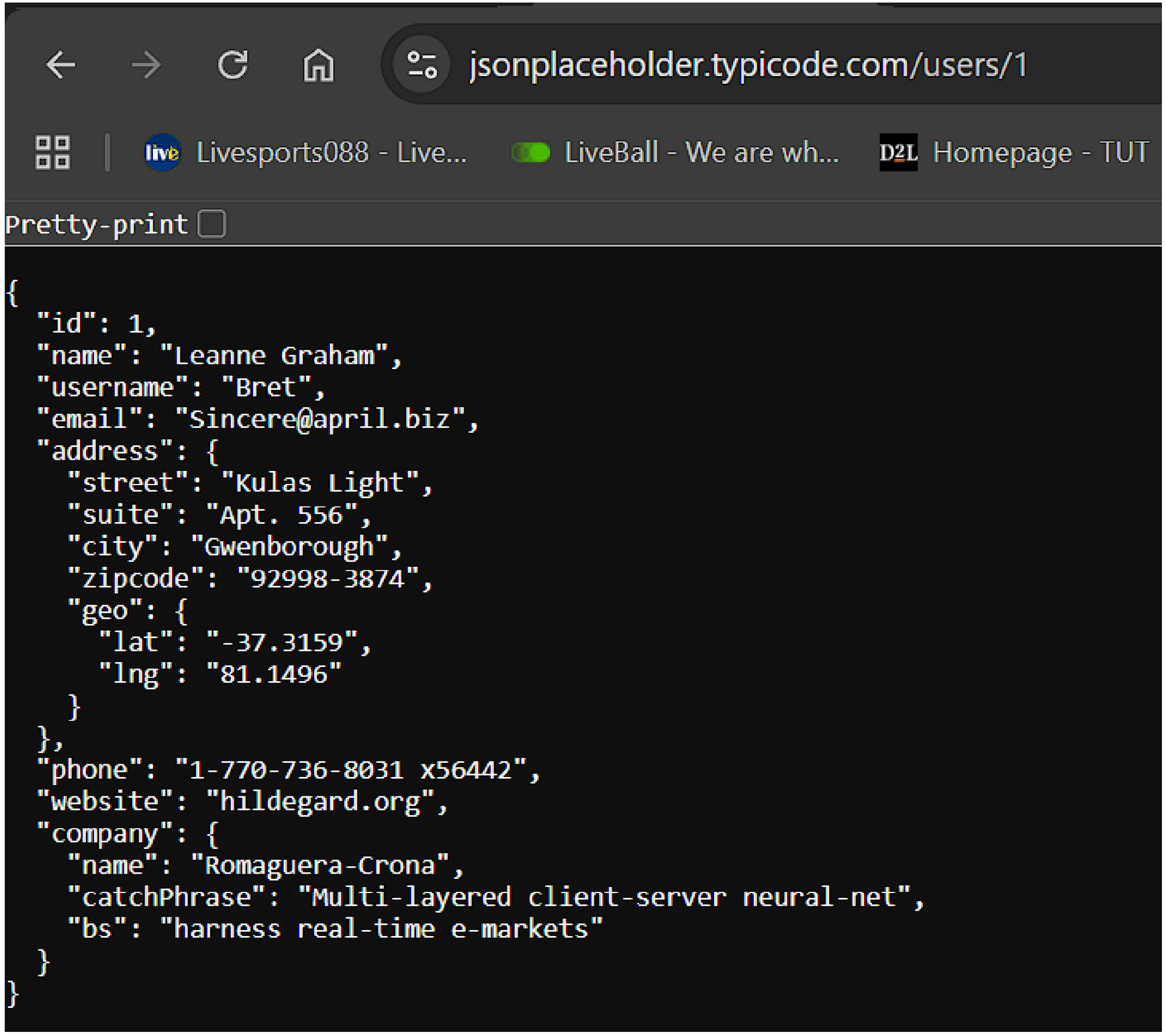


Figure 3: Accessing User 2's profile data simply by changing the URL ID parameter (IDOR).

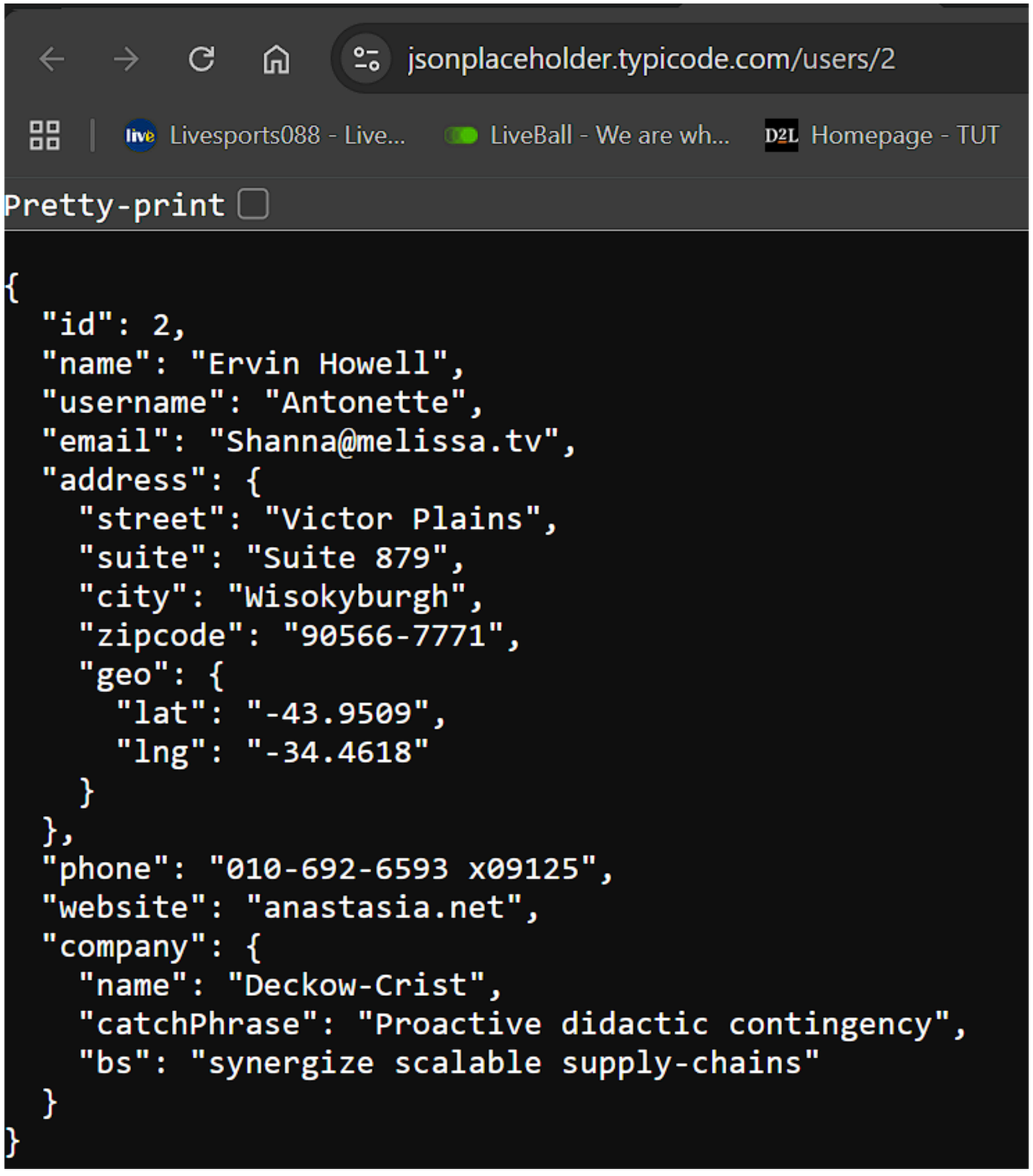
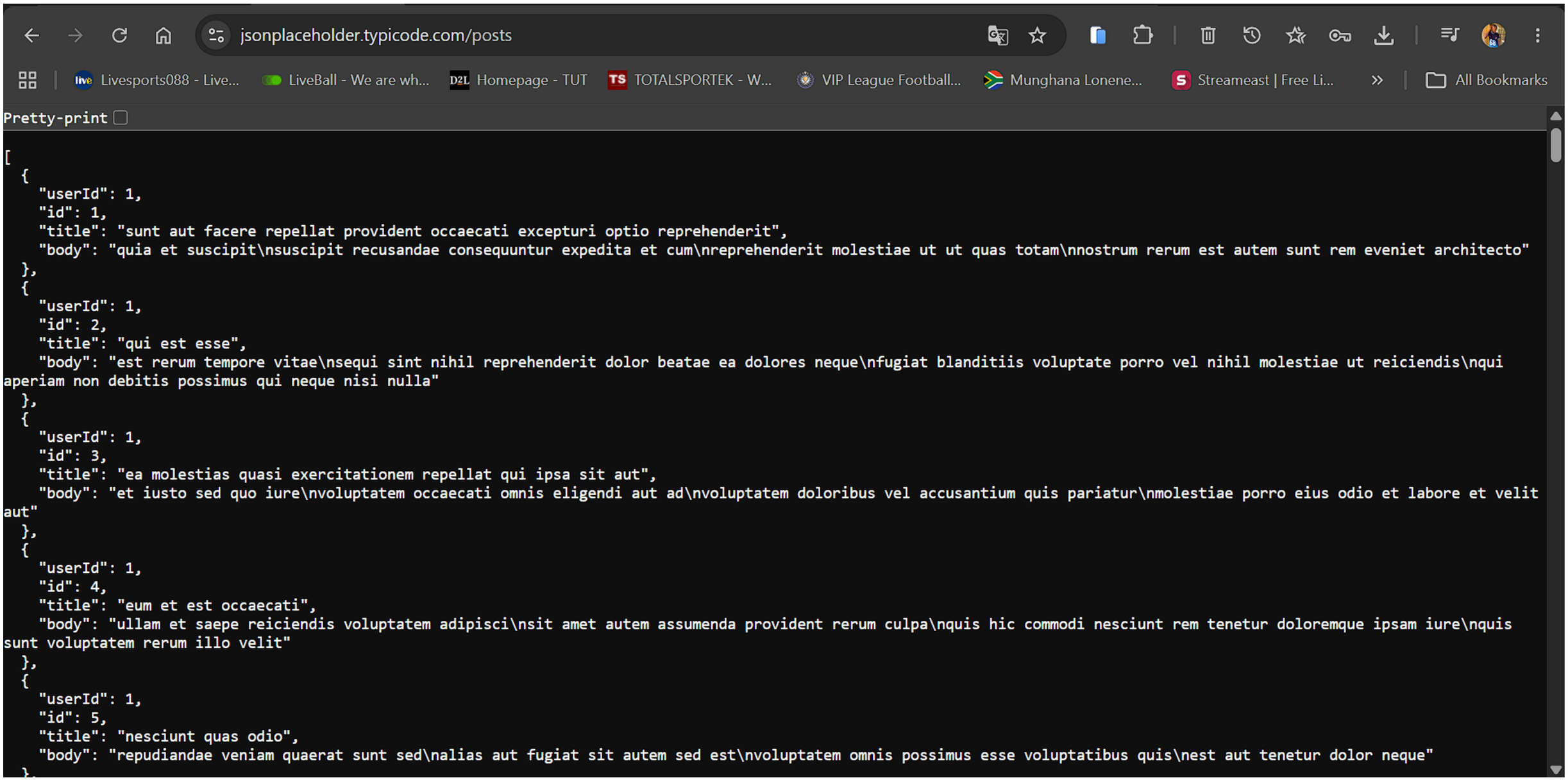


Figure 4: Excessive Data Exposure - API dumps 100 posts in a single response with no pagination.



MORE POSTS

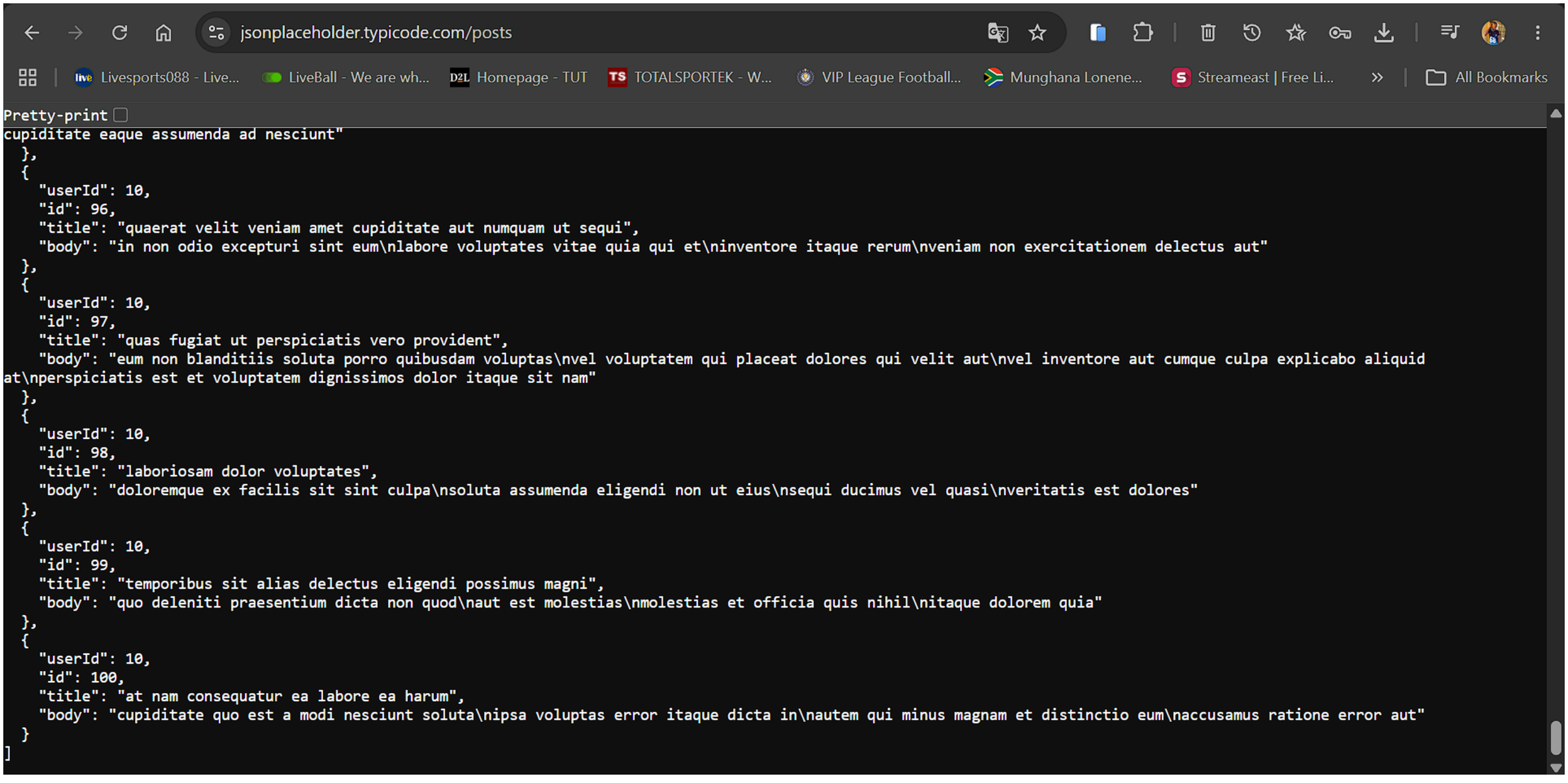

jsonplaceholder.typicode.com/posts
Livesports088 - Live...
LiveBall - We are wh...
Homepage - TUT
TOTALSPORTEK - W...
VIP League Football...
Munghana Lonene...
Streameast | Free Li...
All Bookmarks
Pretty-print
cupiditate eaque assumenda ad nesciunt"
},
{
"userId": 10,
"id": 96,
"title": "quaerat velit veniam amet cupiditate aut numquam ut sequi",
"body": "in non odio excepturi sint eum\nlabore voluptates vitae quia qui et\ninventore itaque rerum\nveniam non exercitationem delectus aut"
},
{
"userId": 10,
"id": 97,
"title": "quas fugiat ut perspiciatis vero provident",
"body": "eum non blanditiis soluta porro quibusdam voluptas\nvel voluptatem qui placeat dolores qui velit aut\nvel inventore aut cumque culpa explicabo aliquid
at\nperspiciatis est et voluptatem dignissimos dolor itaque sit nam"
},
{
"userId": 10,
"id": 98,
"title": "laboriosam dolor voluptates",
"body": "doloremque ex facilis sit sint culpa\nsoluta assumenda eligendi non ut eius\nsequi ducimus vel quasi\nveritatis est dolores"
},
{
"userId": 10,
"id": 99,
"title": "temporibus sit alias delectus eligendi possimus magni",
"body": "quo deleniti praesentium dicta non quod\naut est molestias\nmolestias et officia quis nihil\nitaque dolorem quia"
},
{
"userId": 10,
"id": 100,
"title": "at nam consequatur ea labore ea harum",
"body": "cupiditate quo est a modi nesciunt soluta\nipsa voluptas error itaque dicta in\nautem qui minus magnam et distinctio eum\naccusamus ratione error aut"
}
]

Figure 5: API request made using Hoppscotch (Postman alternative) to GET /users. The response returns user data without requiring an API key.

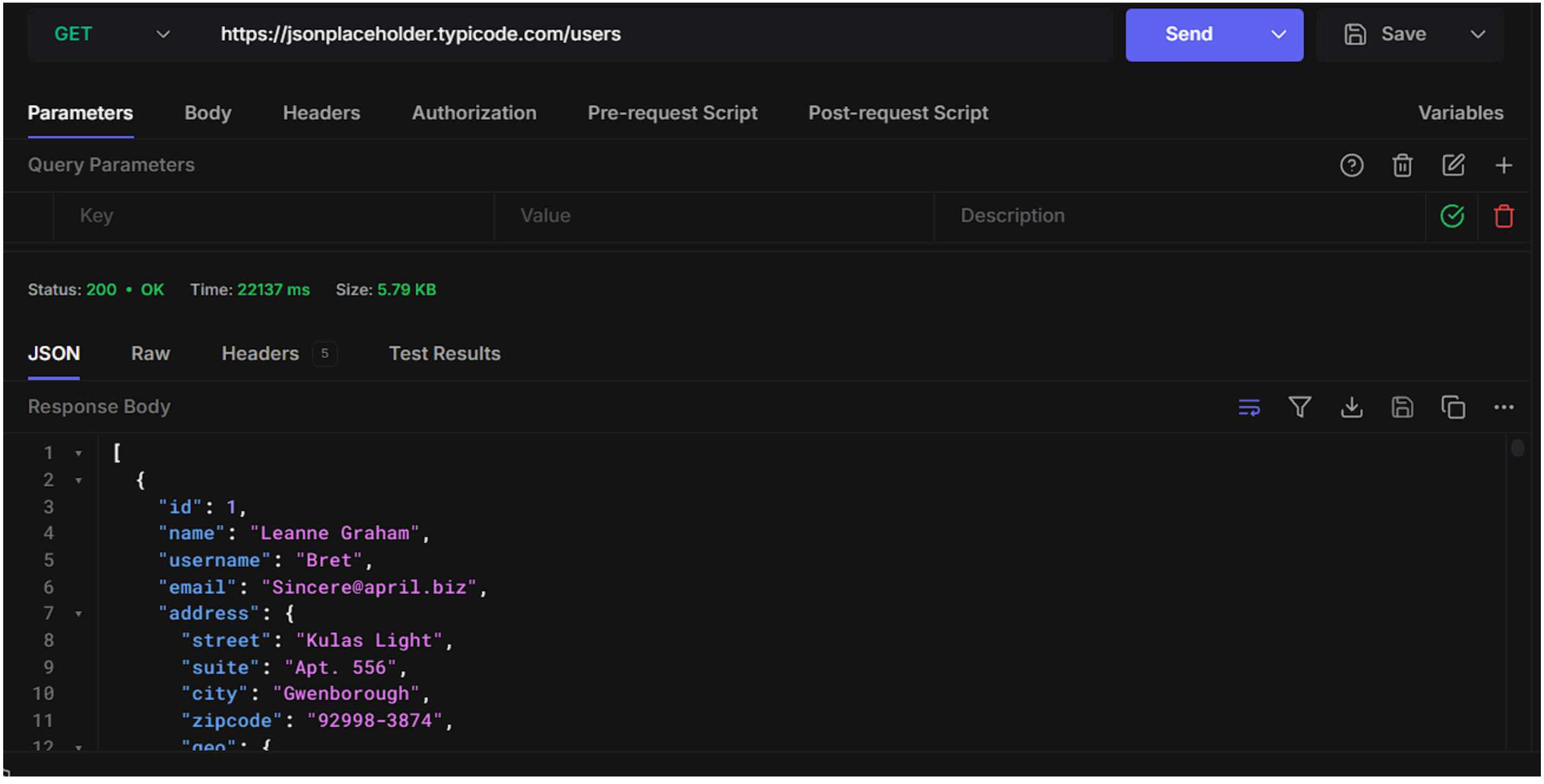


Figure 6: Request to GET /users/2 showing that any user ID can be accessed without authorization (IDOR vulnerability).

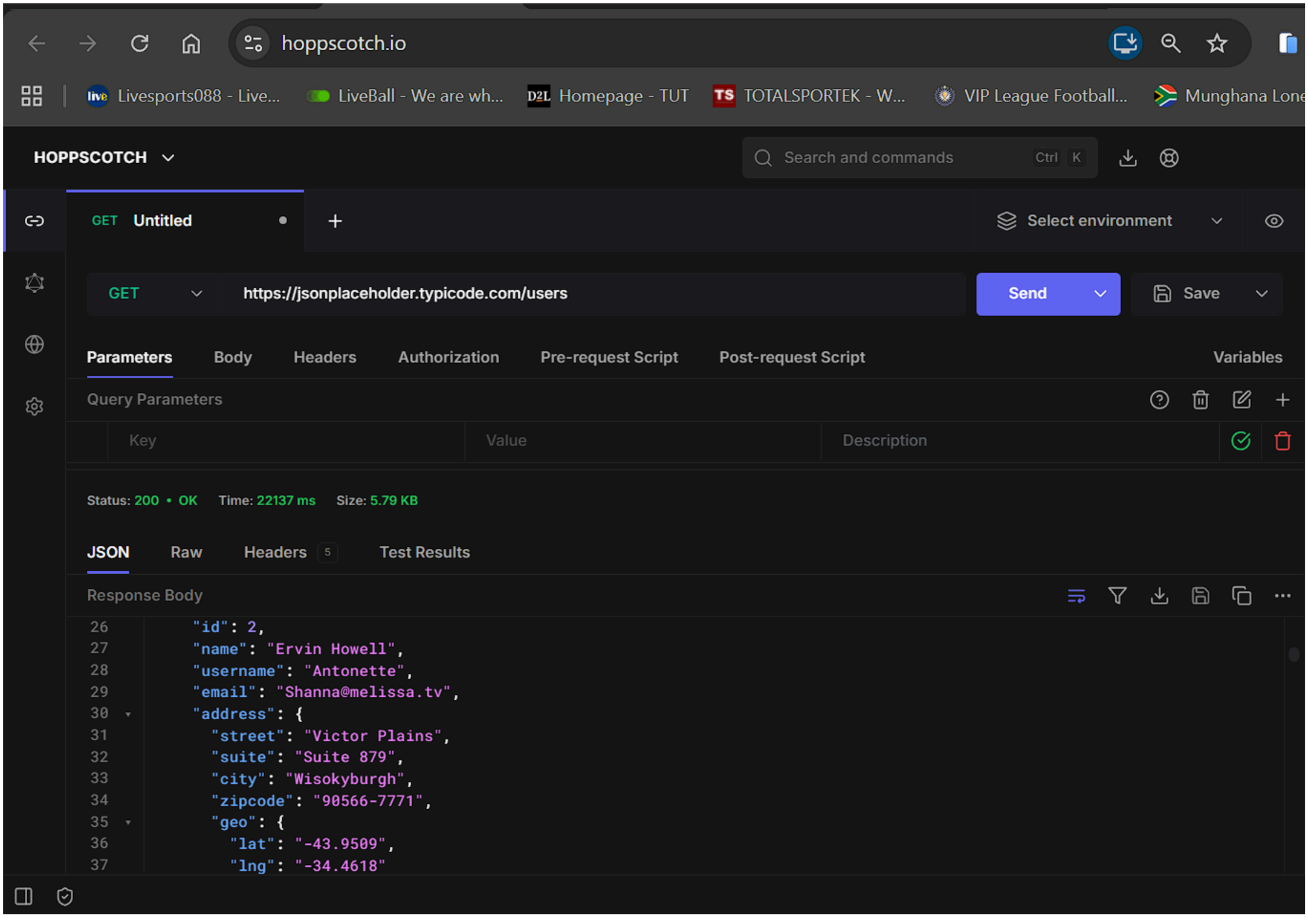


Figure 7: The /users endpoint returns a complete list of all users (including full names, emails, and addresses) in a single request, excessive data exposure.

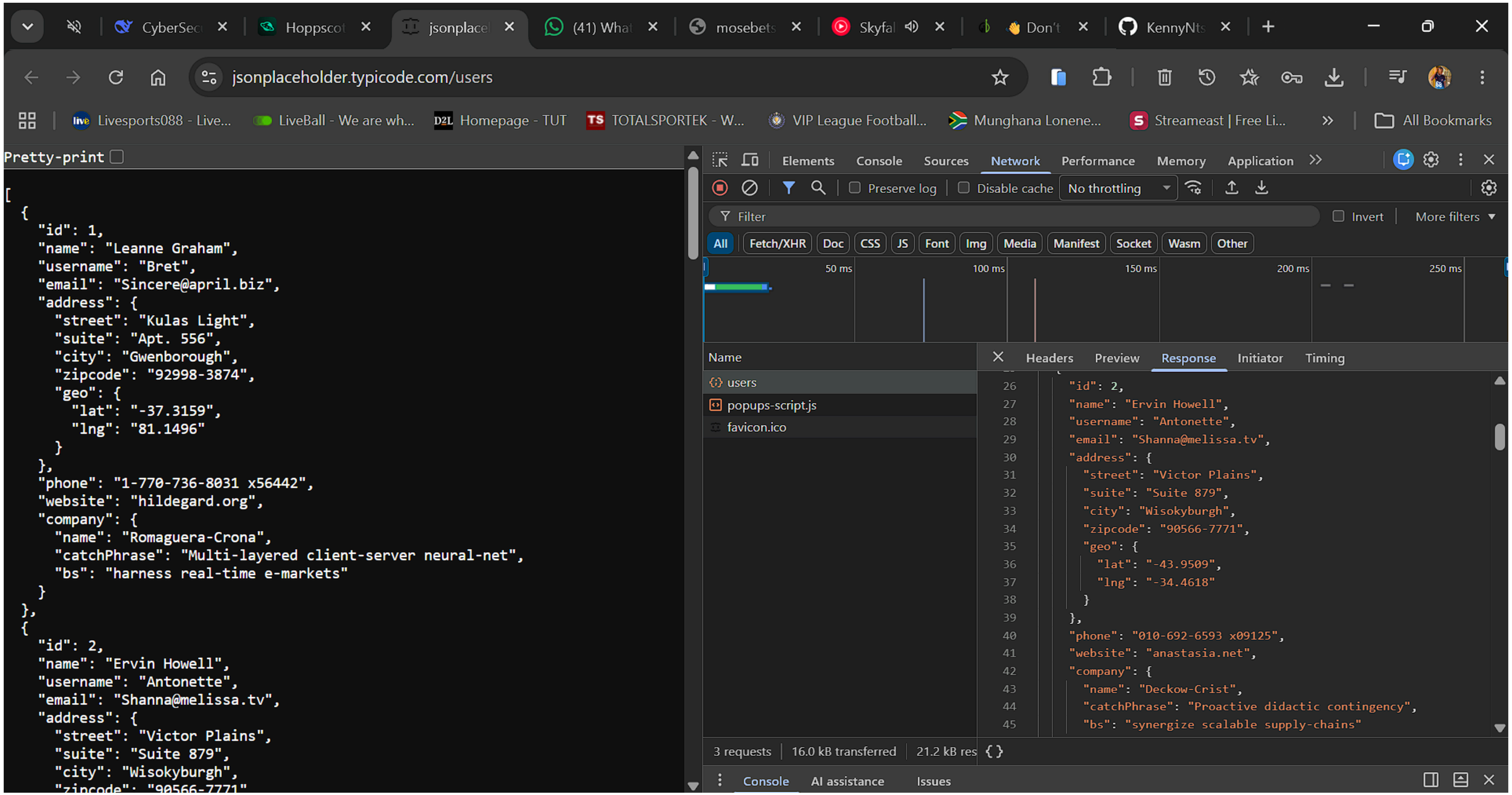
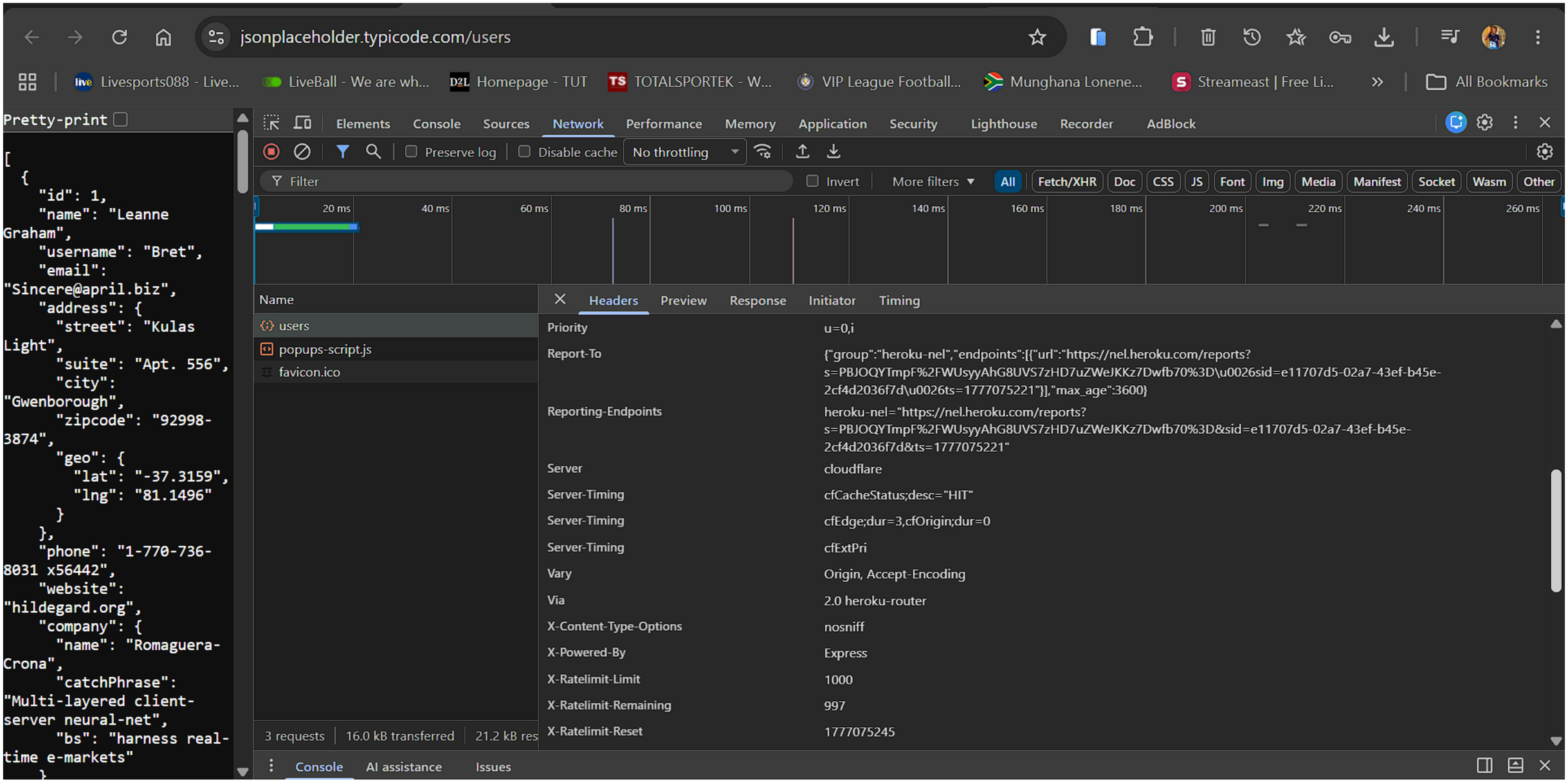


Figure 8: Response headers showing X-Powered-By: Express and Server: heroku. This exposes the underlying technology stack to attackers, aiding reconnaissance.



Conclusion

The API security risk analysis identified critical vulnerabilities in the JSONPlaceholder API, including unauthenticated access, broken authorization (IDOR), excessive data exposure, and server information disclosure. These issues are common in poorly secured SaaS applications and can lead to significant data breaches if implemented in production environments.

By implementing token-based authentication, strict access controls, pagination, and rate limiting, organizations can substantially improve their API security posture and protect sensitive customer data.

Regular API security audits are essential for modern businesses to stay ahead of evolving threats.